

Uso del perceptrón

1. Creación de conjuntos de datos de entrenamiento

Primero define algunos parámetros para generar los datos que se utilizarán para entrenar un perceptrón. Aquí suponemos dos grupos con la etiqueta -1 y 1 y las dos clases están equilibradas (es decir, la cantidad de puntos de datos es la misma en dos grupos). Definimos N como el número de puntos de datos en cada grupo. Las coordenadas (x, y) para cada punto de datos se crean aleatoriamente a partir de una distribución uniforme.

Para el grupo con etiqueta -1, las coordenadas de cada punto de datos se crean aleatoriamente entre 0 y 1. Para el grupo con etiqueta 1, las coordenadas de cada punto de datos se crean aleatoriamente entre (x_offset, 1+x_offset) y (y_offset, 1+y_offset). Un x_offset y y_offset más grandes (por ejemplo, 1.0) **pueden crear un conjunto de datos separable linealmente**, mientras que un x_offset y y_offset más pequeños (por ejemplo, 0.2) pueden crear un conjunto de datos no separable linealmente.

```
> N = 50 # total number of data points each group
> x_offset = 0.5 # group separation on x axis
> y_offset = 0.5 # group separation on y axis
>
> g1_x = runif(N, min = 0, max = 1)
> g1_y = runif(N, min = 0, max = 1)
> g2_x = runif(N, min = 0+x_offset, max = 1+x_offset)
> g2_y = runif(N, min = 0+y_offset, max = 1+y_offset)
> g_x = c(g1_x, g2_x)
> g_y = c(g1_y, g2_y)
> group = c(rep(-1,N), rep(1,N))
>
> print(g_x)
[1] 0.90377544 0.64377375 0.23839604 0.62252560 0.66161241 0.74265393 0.95434974 0.49188888 0.08401021 0.30218392
[11] 0.22245718 0.80297990 0.69183688 0.69640502 0.25515445 0.07923604 0.35852526 0.84219228 0.32198075 0.37294312
[21] 0.23345860 0.47924540 0.88637356 0.51000429 0.33431587 0.09600734 0.22064690 0.35118550 0.37382656 0.41572329
[31] 0.67785693 0.38864449 0.61446091 0.52037914 0.58108586 0.60065203 0.88906963 0.19413305 0.67451996 0.64433543
[41] 0.82966884 0.02875692 0.01906758 0.35096728 0.00254381 0.46004457 0.50358872 0.26011987 0.73324407 0.42702788
[51] 0.64367725 0.71014421 1.21231802 1.42512639 1.20972064 0.97401554 0.63037875 0.76981698 1.24330886 1.40086822
[61] 0.68683066 0.94472566 0.97783587 1.46304215 0.57803936 0.83738786 1.08154007 0.50204993 1.47861778 1.39730306
[71] 1.04319069 0.84260039 0.81189566 0.50815521 0.60676089 0.94687882 1.49011698 1.10767031 1.48710885 1.18205469
[81] 0.74782633 1.04230012 1.48183983 0.56164491 0.62703674 0.82509528 1.12794573 1.31708987 1.11457208 0.75686843
[91] 0.96118177 1.39183414 1.40126998 1.22891885 0.64632117 1.06711912 1.06590398 0.81082021 0.78193518 1.24683223
>
> print(g_y)
```



```

> w0 = 0.1 # initial weight
> w1 = 0.2 # initial weight
> w2 = 0.3 # initial weight
> M = 15      # number of epochs to run
> eta = 0.005 # learning rate
> th = 0.9    # threshold to stop
> verbose = F # whether detailed weight update info is printed

```

3. Modelo del perceptrón

El modelo del perceptrón se define en la siguiente inserción para ciclos con el ciclo exterior para la época y el ciclo interior para la actualización del peso de cada punto de datos. La precisión se calcula después de cada época para decidir si se necesita la próxima época o no. Para cada época, el orden de los puntos de datos para actualizar el peso se baraja aleatoriamente.

```

> for (i in 1:M){
+   print(paste('Epoch starts: ', i))
+   ## We reshuffle the order of the datapoint for each epoch.
+   index = 1:(2*N)
+   index = sample(index)
+
+   for (j in index){
+     y_j = w0 + w1*g_x[j] + w2*g_y[j]
+     if (y_j >= 0){
+       pred_j = 1
+     }else{
+       pred_j = -1
+     }
+
+     w0 = w0 + eta*(group[j] - pred_j)*1.0
+     w1 = w1 + eta*(group[j] - pred_j)*g_x[j]
+     w2 = w2 + eta*(group[j] - pred_j)*g_y[j]
+     if (verbose == T){
+       print(paste(' -> updating data point ', j, ' : '))
+       print(paste('   -> w0: ', w0))
+       print(paste('   -> w1: ', w1))
+       print(paste('   -> w2: ', w2))
+     }
+   }
+
+   y_all = w0 + w1*g_x + w2*g_y
+   y_pred = y_all
+   y_pred[y_all >= 0] = 1
+   y_pred[y_all < 0] = -1
+ }

```

```
+ acc = sum(y_pred == group)/length(group)
+ print(paste('Epoch ends: ', i, ' WITH accuracy: ', acc))
+ if (acc >= th){
+   break
+ }
+ }
```

```
[1] "Epoch starts: 1"
[1] "Epoch ends: 1 WITH accuracy: 0.86"
[1] "Epoch starts: 2"
[1] "Epoch ends: 2 WITH accuracy: 0.85"
[1] "Epoch starts: 3"
[1] "Epoch ends: 3 WITH accuracy: 0.86"
[1] "Epoch starts: 4"
[1] "Epoch ends: 4 WITH accuracy: 0.88"
[1] "Epoch starts: 5"
[1] "Epoch ends: 5 WITH accuracy: 0.86"
[1] "Epoch starts: 6"
[1] "Epoch ends: 6 WITH accuracy: 0.87"
[1] "Epoch starts: 7"
[1] "Epoch ends: 7 WITH accuracy: 0.85"
[1] "Epoch starts: 8"
[1] "Epoch ends: 8 WITH accuracy: 0.87"
[1] "Epoch starts: 9"
[1] "Epoch ends: 9 WITH accuracy: 0.87"
[1] "Epoch starts: 10"
[1] "Epoch ends: 10 WITH accuracy: 0.87"
[1] "Epoch starts: 11"
[1] "Epoch ends: 11 WITH accuracy: 0.83"
[1] "Epoch starts: 12"
[1] "Epoch ends: 12 WITH accuracy: 0.87"
[1] "Epoch starts: 13"
[1] "Epoch ends: 13 WITH accuracy: 0.87"
[1] "Epoch starts: 14"
[1] "Epoch ends: 14 WITH accuracy: 0.86"
[1] "Epoch starts: 15"
[1] "Epoch ends: 15 WITH accuracy: 0.88"
```

4. Comprobación de la predicción

Ahora verifiquemos la predicción para cada punto de datos con el peso final del algoritmo del perceptrón y tracemos el modelo del perceptrón entrenado final (es decir, la línea discontinua en el gráfico).

```
> y_all = w0 + w1*g_x + w2*g_y
> print(y_all)
```

```
[1] 0.032668457 -0.001655655 -0.048559710 0.005204399 -0.050270782 -0.037061111 0.006060346 -0.032930649
[9] -0.067737197 -0.027411169 -0.028351532 -0.002189773 0.001190769 0.009285777 -0.056558265 -0.070503149
[17] -0.049138101 0.007166994 -0.035637848 -0.042716489 -0.091745666 -0.025831275 0.005741935 -0.040335706
[25] -0.086492788 -0.086945333 -0.034228498 -0.045331828 -0.052089579 -0.032455068 -0.037121197 -0.015879930
[33] -0.063410109 -0.056537397 -0.059008289 -0.049566149 0.016867337 -0.099760451 -0.023150529 -0.015802492
[41] -0.005884122 -0.055404048 -0.062513829 -0.057806732 -0.114286107 -0.041349696 -0.065567279 -0.094020020
[49] -0.016870593 -0.039751982 0.027156423 -0.014188358 0.061856363 0.093037054 0.045832180 0.005058431
[57] 0.022681286 0.049467048 0.050630389 0.049411262 0.014386204 0.008180512 0.024377760 0.100897054
[65] 0.025773274 0.060834015 0.022119340 0.001950445 0.062966177 0.083640588 0.037210443 0.031657937
[73] 0.009841040 0.025965360 0.040844228 0.004301514 0.107185311 0.036847762 0.089271614 0.093104098
[81] 0.003993986 0.044527439 0.101119527 -0.024385200 -0.004119073 0.055165128 0.051683951 0.038231562
[89] 0.063645961 0.002494704 0.055235804 0.094082879 0.102179376 0.062936584 0.016902036 0.064902464
[97] 0.080563542 -0.007753917 0.032600464 0.079981323
```

```
> y_pred = y_all
> y_pred[y_all >= 0] = 1
> y_pred[y_all < 0] = -1
```

```
> print(y_pred)
```

```
[1]  1 -1 -1  1 -1 -1  1 -1 -1 -1 -1 -1  1  1 -1 -1 -1  1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
[37]  1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1  1 -1  1  1  1  1  1  1  1  1  1  1  1  1  1
[73]  1  1  1  1  1  1  1  1  1  1  1 -1 -1  1  1  1  1  1  1  1  1  1 -1  1  1
```

```
> acc = sum(y_pred == group)/length(group)
```

```
> print(acc)
```

```
[1] 0.88
```

```
> plot(g_x, g_y, type='n', xlab='X', ylab='Y')
```

```
> points(g1_x, g1_y, col='red')
```

```
> points(g2_x, g2_y, col='blue')
```

```
> abline(a = -1.0*w0/w2, b = -1.0*w1/w2, col='dark green', lwd=3, lty=2)
```

